

dextents Index Type Parameter

Document #:	P2389R2
Date:	2024-06-24
Project:	Programming Language C++ LWG
Reply-to:	Bryce Adelstein Lelbach Mark Hoemmen

Contents

- 1 Authors
- 2 Abstract
- 3 Revision history
 - 3.1 R0
 - 3.2 R1
 - 3.3 R2
- 4 Background
- 5 Problem
- 6 Proposed Changes
- 7 Note on freestanding
- 8 Implementation
- 9 Wording

§ 1 Authors

- Bryce Adelstein Lelbach (he/him/his), NVIDIA, brycelelbach@gmail.com
- Mark Hoemmen (he/him/his), NVIDIA, mhoemmen@nvidia.com

§ 2 Abstract

[P2299R3](#) added dextents to make it less verbose to use `mdspan` in common cases. Later, an index type parameter was added to dextents, which increases its verbosity, defeating

its original purpose. We should fix this.

§ 3 Revision history

§ 3.1 R0

R0 published 2024-02-15 and reviewed by LEWG 2024-03-05. Poll results follow.

POLL: We want to make a breaking change to `dextents` to fix the issue presented in P2389R0.

Results (SF/F/N/A/SA): 0/8/6/4/1.

- Attendance: 25
- Author's Position: WF
- Outcome: No consensus for a change

POLL: Vote once for your preferred name option:

- `std::dexts`: 0 votes
- `std::dms`: 11 votes
- Please explore a new name (in the reflector): 2 votes

POLL: Forward P2389R1 to LWG for C++26 (which will be P2389R0 modified to add a new facility (`std::dms`) as suggested, but with the last param as the type defaulted to `std::size_t`) (to be confirmed with a library evolution electronic poll).

Results (SF/F/N/A/SA): 5/8/2/0/0.

- Attendance: 26
- Author's Position: SF
- Outcome: Strong consensus in favor

§ 3.2 R1

R1, to be published 2024-03-12, includes the following changes.

- Discuss and implement the above LEWG poll decisions
- Add wording
- Discuss freestanding implications (we think there are none)
- Add link to implementation
- Correct use of `extents` in nonwording sections

§ 3.3 R2

LWG reviewed R1 at the St. Louis meeting on 2024-06-24. There, LWG asked the authors to fix formatting by replacing `@ ... _@` code-font text with actual italics. The authors made that change.

§ 4 Background

P0009R18 added `mdspan`, a non-owning multidimensional span abstraction to the C++ Standard Library. It is excellent and flexible, allowing users to customize data layout, access method, and index type. However, this flexibility often comes with verbosity.

The length of each dimension (the extent) of an `mdspan` are represented by an `extents` object, and each extent may be expressed either statically or dynamically. Use cases based on P0009R18's wording would look like this.

```
// All static extents
mdspan<float, extents<64, 64, 64>> a(d);

// All dynamic extents
mdspan<float, extents<dynamic_extent, dynamic_extent,
dynamic_extent>> a(d, 64, 64, 64);

// Mixed static and dynamic extents
mdspan<float, extents<64, dynamic_extent, 64>> a(d, 64);
```

P2299R3 sought to improve `mdspan`'s usability for one of the most common cases: when all extents are dynamic. First, it added deduction guides to make class template argument deduction (CTAD) work for `mdspan` and `extents`.

```
std::vector<float> storage(64 * 64);
mdspan a(storage.data(), 64, 64); // All dynamic.
```

However, CTAD does not help in all situations. For example, it cannot be used when declaring a class member or a function parameter.

```
struct X {
    std::mdspan<float, std::extents<std::dynamic_extent,
std::dynamic_extent, std::dynamic_extent>> a;
};

void f(std::mdspan<float, std::extents<std::dynamic_extent,
std::dynamic_extent, std::dynamic_extent>> a);
```

To simplify these cases, P2299R3 also added `dextents<N>`, a template alias for an `extents` with `N` dynamic extents.

```
template <std::size_t N>
using dextents = /* ... */;

struct X {
    std::mdspan<float, std::dextents<3>> a;
};

void f(mdspan<float, std::dextents<3>> a);
```

§ 5 Problem

Originally, `mdspan` and `dextents` used a fixed index type (`std::size_t`). However, the signedness and size of the index type can affect performance in certain cases. So, [P2553R1](#) parameterized the index type used by `mdspan` and `dextents`. As a part of this change, an index type template parameter was added to `dextents`.

```
template <class IndexType, std::size_t Rank>
using dextents = /* ... */
```

This change has made using `dextents` more verbose, which is unfortunate, as the main purpose of `dextents` was to make common uses as simple as possible.

```
struct X {
    std::mdspan<float, std::dextents<std::size_t, 3>> a;
};

void f(mdspan<float, std::dextents<std::size_t, 3>> a);
```

Index type customization is an important feature for `mdspan` to support, but it is not something that most users will need to use or think about. If they do need it, they can always use the more verbose `dextents`.

§ 6 Proposed Changes

In R0, we originally proposed removing the index type parameter from `dextents` and making it always use `size_t` as the index type. For example, instead of typing `dextents<int, 3>` as an alias for `dextents<int, dynamic_extent, dynamic_extent, dynamic_extent>`, users would type `dextents<3>`.

This would have been a source-breaking change. As of the publication date, MSVC's STL and LLVM's `libc++` are already shipping `dextents`. GCC's `libstdc++` is not shipping `mdspan` yet. However, since `dextents` is a template alias, it would have had no ABI impact.

LEWG reviewed R0 and voted against the breaking change. Instead, LEWG approved our alternative design: leave `dextents` alone, but add a new `dims` template alias. LEWG asked us to adjust the `dims` design by adding an index type template parameter as its *last* template parameter, and defaulting it to `size_t`. The resulting alias looks like this.

```
template<size_t Rank, class IndexType = size_t>
using dims = dextents<IndexType, Rank>;
```

Before adding this alias, the above use cases where CTAD could not be used would look like this.

```
struct X {
    // Member declaration
    std::mdspan<float, std::dextents<std::size_t, 3>> a;
};

// Function parameter
void f(std::mdspan<float, std::dextents<std::size_t, 3>> a);
```

After adding this alias, those two use cases would look like this.

```
struct X {
    // Member declaration
    std::mdspan<float, std::dims<3>> a;
};

// Function parameter
void f(mdspan<float, std::dims<3>> a);
```

The result is even more compact than dextents. It also abbreviates a more familiar word, “dimensions.”

§ 7 Note on freestanding

[P2833R2](#), adopted in Kona 2023, added an `// all freestanding` comment to the beginning of the [\[mdspan.syn\]](#) synopsis. In our view, our proposal would not change this freestanding status.

§ 8 Implementation

We have implemented the proposal as [Pull Request 324](#) in the reference mdspan implementation.

§ 9 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

Make the following changes to the latest C++ Working Draft as of the time of writing. All wording is relative to the latest C++ Working Draft.

In [\[version.syn\]](#), increase the value of the `__cpp_lib_mdspan` macro by replacing `YYMML` below with the integer literal encoding the appropriate

year (YYYY) and month (MM).

```
#define __cpp_lib_mdspan YYYYMMML // also in <mdspan>
```

In the `<mdspan>` header synopsis [`mdspan.syn`], after the declaration of `dextents`, add the following declaration of `dims`.

```
// [mdspan.extents.dims], alias template `dims`  
template<size_t Rank, class IndexType = size_t>  
    using dims = see below;
```

Immediately after section [`mdspan.extents.dextents`], insert a new section “Alias template `dims`” [`mdspan.extents.dims`] with the following wording.

```
template<size_t Rank, class IndexType = size_t>  
    using dims = see below;
```

Result: A type `E` that is a specialization of `extents` such that `E::rank() == Rank` && `E::rank() == E::rank_dynamic()` is true, and `E::index_type` denotes `IndexType`.